

Advanced Deep Learning 2024

Assignment 1

Aidin Sarfaraz <pgq852@alumni.ku.dk>
Kaizheng Li (Jackie) <pkd916@alumni.ku.dk>
Martin Vigholt <zwc822@alumni.ku.dk>

November 29, 2024

Contents

1	Backpropagation pen and paper	2
1.1	C	2
1.2	J	2
2	AutoDiff framework	4
2.1	B	4
2.1.1		4
2.1.2		4
2.1.3		5
2.1.4		5
2.2	L	6
2.2.1	Over fitting	6
2.2.2	Under fitting	7
2.2.3	Just right fitting	8
3	Deep versus wide networks	9
3.1	Question 1	9
3.2	Question 2	9
3.3	Question 3	12
4	MLOps	15
4.1	Question 1	15
4.2	Question 2 and Question 3	15

1 Backpropagation pen and paper

1.1 C

$$y_j = z_j^{(L)} \quad (1)$$

$$z_j^{(l)} = h_l(a_j^{(l)}) \quad l = 1, \dots, L \quad (2)$$

$$a_j^{(l)} = \sum_i^{D_l} w_{ji}^{(l)} \cdot z_j^{(l-1)} \quad l = 2, \dots, L \quad (3)$$

$$a_j^{(1)} = \sum_i^{D_1} w_{ji}^{(1)} \cdot x_i \quad (4)$$

Where D_l is the number of inputs for the l 'th layer

1.2 J

$$\frac{\partial E(w)}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} z_i^{(l-1)} \quad (5)$$

$$\delta_j^{(l)} = \sum_{k=1}^K \delta_k^{(l+1)} w_{kj}^{(l+1)} h'_l(a_j^{(l)}) \quad (6)$$

The general backpropagation rule for $l < L$ is simply an application of the chain rule

$$f(g(x))' = f'(g(x)) \cdot g'(x) = \frac{\partial f(g(x))}{\partial g(x)} \cdot \frac{\partial g(x)}{\partial x}$$

Where

$$E_i = h^{(L)} \left(\sum_{j=1}^{d^{(L-1)}} \left(w_{ji}^{(L)} \cdot h_j^{(L-1)} \left(a_j^{(L-1)} \right) \right) \right) \quad (7)$$

$$f(g(x)) = E(x) \quad (8)$$

$$g(x) = \sum_{j=1}^{d^{(L-1)}} \left(w_{ji}^{(L)} \cdot h_j^{(L-1)} \left(a_j^{(L-1)} \right) \right) \quad (9)$$

$$x = w_{ji}^{(L)} \quad (10)$$

The formula states that we can find the partial derivative of the final loss function w.r.t an individual weight, w_{ji} by multiplying its input (activation from layer before) by the sum of partial derivatives of the loss function w.r.t "all the connections" from the neurons w_{ji} activates into and all the way up to the final loss.

The terms represent:

- $\delta_k^{(l+1)}$: The overall contribution to the final loss by all neurons upstream that are connected to a neuron in the current layer.
- $w_{kj}^{(l+1)}$: Scales the share of each upstream neuron in the total loss by its weight
- $h'_l(a_j^{(l)})$: Represents the final loss' sensitivity to changes in the current neuron.

Putting it all together, the expression is the computation of the change rate in the final loss w.r.t a given weight, w_{ji} , by multiplying how much each neuron connected to w_{ji} downstream contributes to the error, scaled by the strength of their connections, and how sensitive w_{ji} 's activation is to changes at all.

There is also a exceptional case when $l = 1$. The formula would be

$$\frac{\partial E(w)}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} x_i \quad (11)$$

$$\delta_j^{(1)} = \sum_{k=1}^K \delta_k^{(2)} w_{kj}^{(2)} h'_1(a_j^{(1)}) \quad (12)$$

$$= \sum_{k=1}^K \delta_k^{(2)} w_{kj}^{(2)} h'_1\left(\sum_i^{D_1} w_{ji}^{(1)} \cdot x_i\right) \quad (13)$$

The terms represent:

- x_i is the input of neural network. i is the feature number of the input.

2 AutoDiff framework

2.1 B

2.1.1

See calculations for expressions below

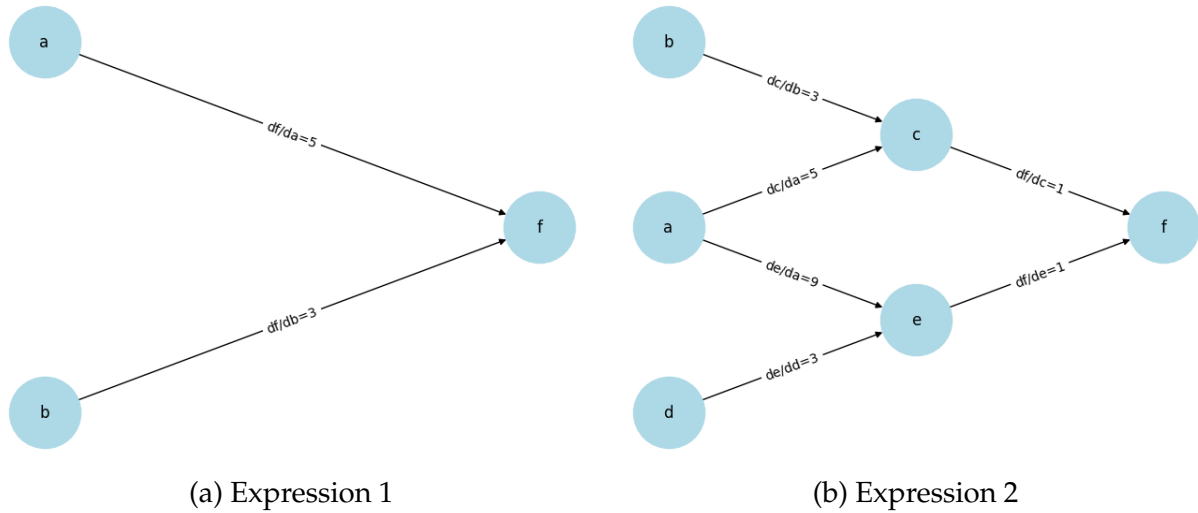


Figure 1: Graph over expressions

2.1.2

Firstly, a and b are initialized with an empty "grad_fn", meaning they are independent / input or leaf nodes. Then, f is defined and initialized by an implicit call to the `__mul__()` function, making f dependent on a and b :

1. $a = \text{Var}(3, [])$
 - $\text{val} = 3.0$
 - $\text{grad_fn} = []$
 - $\text{grad} = 0.0$
2. $b = \text{Var}(5, [])$
 - $\text{val} = 5.0$
 - $\text{grad_fn} = []$
 - $\text{grad} = 0.0$
3. $f = a * b = \text{Var}(\text{self.v} * \text{other.v}, \text{lambda: } [(\text{self}, \text{other.v}), (\text{other}, \text{self.v})])$
 - $\text{val} = 3.0 * 5.0 = 15.0$
 - $\text{grad_fn} = [(a, 5.0), (b, 3.0)]$
 - $\text{grad} = 0.0$

2.1.3

f differentiated with respect to its dependencies:

1. Expression:

$$\frac{\partial f}{\partial a} = \frac{\partial(a \cdot b)}{\partial a} = 1 \cdot b = 5 \quad (14)$$

$$\frac{\partial f}{\partial b} = \frac{\partial(a \cdot b)}{\partial b} = a \cdot 1 = 3 \quad (15)$$

$$\frac{\partial f}{\partial f} = 1 \quad (16)$$

2. Expression:

$$\frac{\partial f}{\partial a} = \frac{\partial(c + e)}{\partial a} = \frac{\partial((a \cdot b) + (a \cdot d))}{\partial a} = 1 \cdot b + 1 \cdot d = 5 + 9 = 14 \quad (17)$$

$$\frac{\partial f}{\partial b} = \frac{\partial(c + e)}{\partial b} = \frac{\partial((a \cdot b) + (a \cdot d))}{\partial b} = a \cdot 1 + 0 \cdot 0 = 3 + 0 = 3 \quad (18)$$

$$\frac{\partial f}{\partial c} = \frac{\partial(c + e)}{\partial c} = 1 + 0 = 1 \quad (19)$$

$$\frac{\partial f}{\partial d} = \frac{\partial(c + e)}{\partial d} = \frac{\partial((a \cdot b) + (a \cdot d))}{\partial d} = 0 \cdot 0 + a \cdot 1 = 0 + 3 = 3 \quad (20)$$

$$\frac{\partial f}{\partial e} = \frac{\partial(c + e)}{\partial e} = 0 + 1 = 1 \quad (21)$$

$$\frac{\partial f}{\partial f} = 1 \quad (22)$$

2.1.4

We have illustrated the sequence of calls to backdrop for the **second** expression (starting with the initial call to backward). The area surrounded by a red square is essentially the sequence of calls for the first, smaller network expression where only f has been replaced by c.

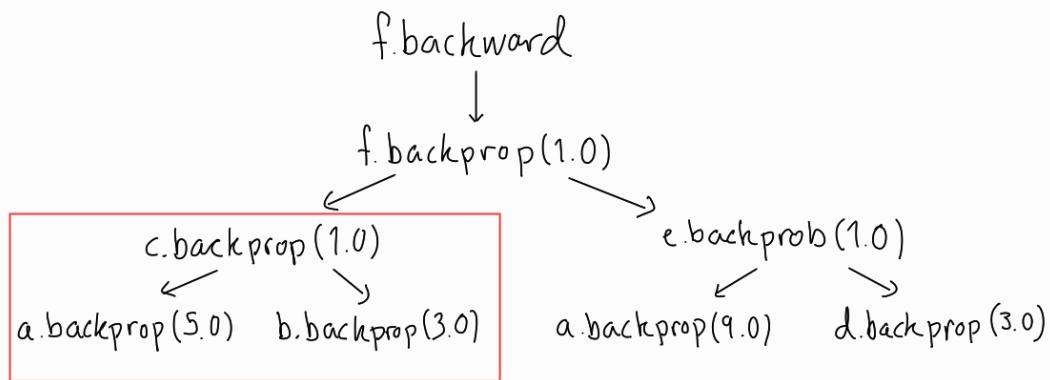


Figure 2: Enter Caption

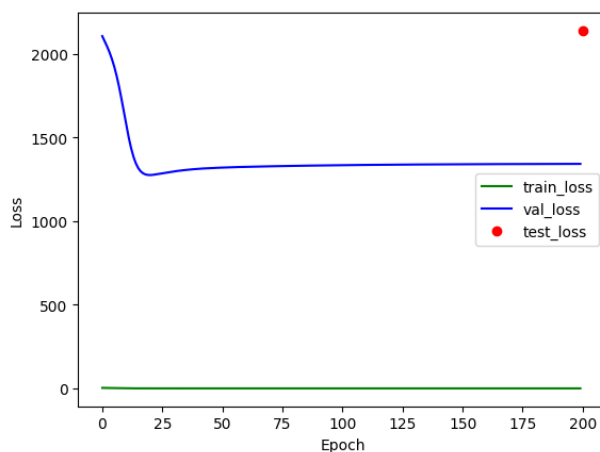
2.2 L

2.2.1 Over fitting

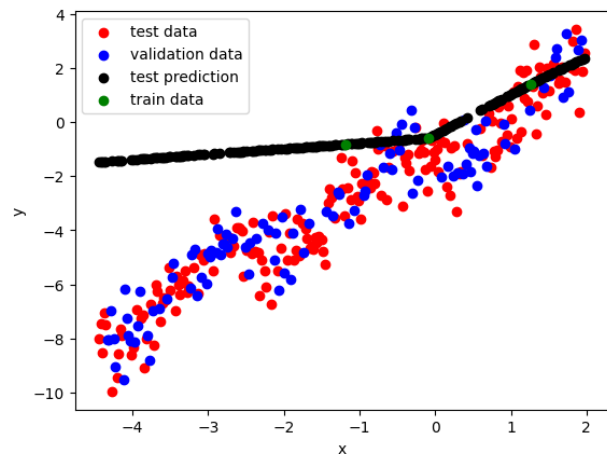
Reasons for overfitting:

- Not enough training data / too much test data
- Overly complex model, neural network too wide and or deep
- Too many epochs without early stopping

```
EPOCHS = 200
LEARN_R = 5e-2
D1 = True
NN_over_fit = [
    DenseLayer(1, 20, lambda x: x.relu()),
    DenseLayer(20, 1, lambda x: x.identity())
]
draw(NN_over_fit, data_generator(noise=0.5, n_samples=300, trainsplit
    =0.01, valsplitt=0.35, D1=D1))
```



(a) Squared loss



(b) Data

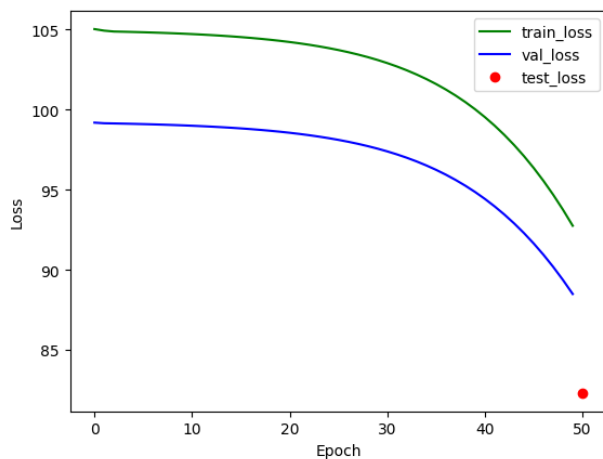
Figure 3: Over fitting

2.2.2 Under fitting

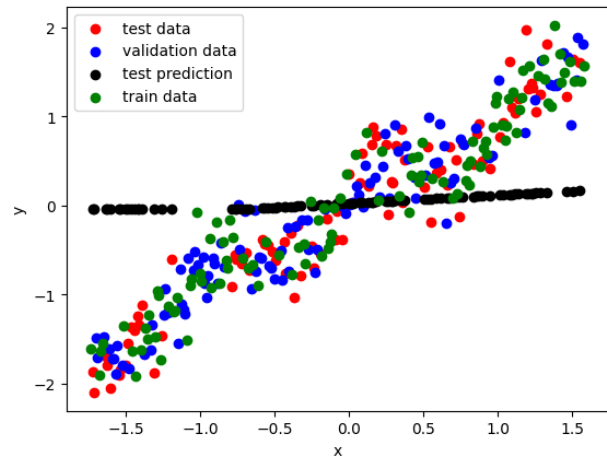
Reasons for underfitting:

- Learning rate too low
- Not enough epochs / too early stopping
- Too simple model, neural network not wide and or deep enough

```
EPOCHS = 50
LEARN_R = 4e-4
D1 = True
NN_under_fit = [
    DenseLayer(1, 2, lambda x: x.relu()),
    DenseLayer(2, 1, lambda x: x.identity())
]
draw(NN_under_fit, data_generator(noise=0.5, n_samples=300, trainsplit
    =0.35, valsplit=0.35, D1=D1))
```



(a) Squared loss

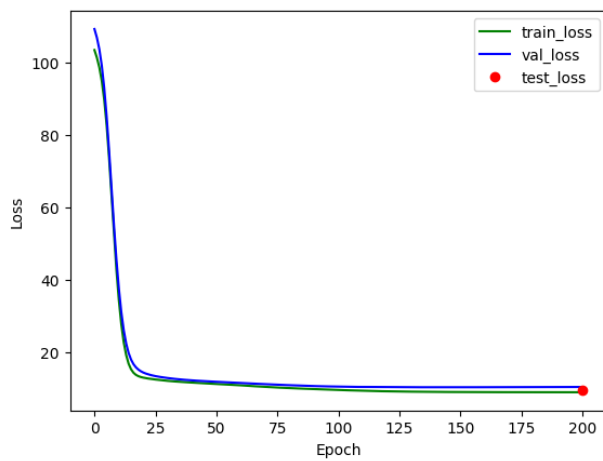


(b) Data

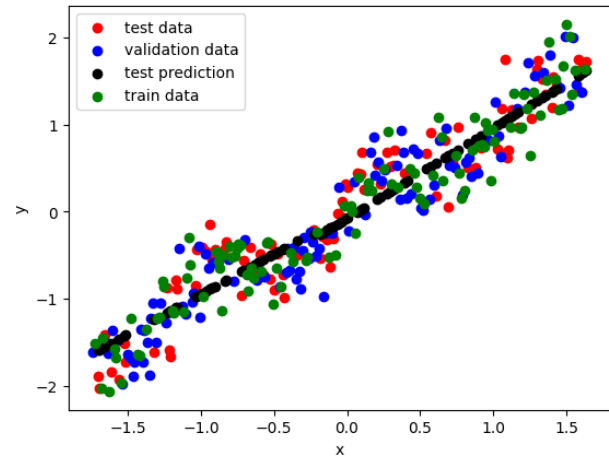
Figure 4: Under fitting

2.2.3 Just right fitting

```
EPOCHS = 200
LEARN_R = 2e-3
D1 = True
NN_just_right_fit = [
    DenseLayer(1, 5, lambda x: x.relu()),
    DenseLayer(5, 1, lambda x: x.identity())
]
draw(NN_just_right_fit, data_generator(noise=0.5, n_samples=300,
    trainsplit=0.35, valsplitt=0.35, D1=D1))
```



(a) Squared loss



(b) Data

Figure 5: Just right fitting

3 Deep versus wide networks

All the code for question 3 and question 4 is stored in `feedforwardAssignment.ipynb`.

3.1 Question 1

There are two versions of the binary cross entropy:

```
loss_fn1 = nn.BCEWithLogitsLoss()
loss_fn2 = nn.BCELoss()
```

`loss_fn1` receives the last layer of output using equation 24 and passes the output of equation 24: $\sigma(\hat{y}_i)$ to equation 23. In contrast, `loss_fn2` only uses equation 23 and equation 23 should only receive $\hat{y}_i \in [0, 1]$

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (23)$$

$$\sigma(\hat{y}_i) = \frac{1}{1 + e^{-\hat{y}_i}} \quad (24)$$

In our training process, we defined `loss_fn1` as our loss function and it works. This is because the output of neural networks are logits and they should be transformed to a range between 0 and 1 before accepting equation . `loss_fn1` automatically helps us do both.

```
FeedForwardNN(
    (model): Sequential(
      (0): Linear(in_features=2, out_features=3, bias=True)
      (1): Tanh()
      (2): Linear(in_features=3, out_features=1, bias=True)
      (3): Identity()
    )
)
```

If we want to use `loss_fn2`, we can specify our neural network as follows:

```
FeedForwardNN(
    (model): Sequential(
      (0): Linear(in_features=2, out_features=3, bias=True)
      (1): Tanh()
      (2): Linear(in_features=3, out_features=1, bias=True)
      (3): Sigmoid()
    )
)
```

3.2 Question 2

As the minimum try, we test all combinations of $\text{depth} \in [0, 1, 2, 3]$ and $\text{width} \in [1, 2, 3]$.

Firstly, we clarify some of the network layout:

- When network depth = 0, the input x_1 and x_2 directly go into y_1 . Therefore, there are no hidden layers, and network width does not affect the layout.

```
if depth == 0:
    model = nn.Sequential(nn.Linear(input_size, output_size))
```

- We do not change the activation function we use in question 1, which means that we have only one *Tanh()* for activation, one hidden layer, and only adding the *Identity()* function. Also, we use *nn.BCEWithLogitsLoss()* as our loss function with a sigmoid activation function at the end.
- When network depth ≥ 2 , we get more than 1 hidden layer. We specify the same width to all hidden layers.

It means that we are not testing something like the following network:

```
FeedForwardNN(
    (model): Sequential(
      (0): Linear(in_features=2, out_features=3, bias=True)
      (1): Tanh()
      (2): Linear(in_features=3, out_features=2, bias=True)
      (3): Identity()
      (4): Linear(in_features=2, out_features=2, bias=True)
      (5): Identity()
      (6): Linear(in_features=2, out_features=1, bias=True)
      (7): Identity()
    )
)
```

Secondly, we think that the mean and standard deviation of the resulting loss function on new dataset is the loss on test set. But the mean and standard deviation are ambiguous, so here's a clarification:

1. We understand the mean of the test loss for one model structure as the average performance of test set loss over 10 random initialization of model with different weights. (All models use the same data set)
2. We understand the mean of test loss for one model structure as the average performance of test set loss over 10 randomly generated datasets

In our implementation, we combined two idea because we are testing the general performance on different structures of neural network.

- It is not reasonable to test 10 random initializations of the same model on the same dataset because we already tested it and found that all the test losses remained similar because similar initialization of the models does not affect how the models converge to training set.

- It is also not good to test 10 different datasets with only one initialization of the neural network.

Finally, here is the description of our setting:

- In total, we have 12 different neural networks or model structures, and three of them are the same with hidden layer depth = 0.
- For each model structure, we randomly generate one dataset (comprising of training set, validation set, and test set) and initiate specific a model structure.
- We first train the model using 20 epochs and learning rate of 0.01. And we use the optimizer "Adam" and loss function "BCEWithLogitsLoss".
- Then we use the trained model to get the test loss. We repeat this 20 times for each model structure.

Here are the tables 1 and the table 2 to record depth and width of each neural network and their corresponding average test loss and standard deviation, mean test accuracy of the loss on test set:

Table 1: Summary of Mean Test Metrics

Depth	Width	Mean Test Loss	Test Loss Std Dev	Mean Test Accuracy
0	1	0.6970	0.0042	0.4620
0	2	0.6994	0.0054	0.4695
0	3	0.6960	0.0039	0.4830
1	1	0.4716	0.0419	0.7580
1	2	0.3547	0.0229	0.7355
1	3	0.0008	0.0033	1.0000
2	1	0.4826	0.0252	0.7435
2	2	0.0126	0.0508	0.9990
2	3	0.0016	0.0057	0.9995
3	1	0.4836	0.0372	0.7555
3	2	0.0061	0.0261	0.9990
3	3	0.0000	0.0000	1.0000

Table 2: Summary of Mean Test Metrics

Depth	Width	Mean Test Loss	Test Loss Std Dev	Mean Test Accuracy
0	1	0.6970	0.0042	0.4620
1	1	0.4716	0.0419	0.7580
2	1	0.4826	0.0252	0.7435
3	1	0.4836	0.0372	0.7555
0	2	0.6994	0.0054	0.4695
1	2	0.3547	0.0229	0.7355
2	2	0.0126	0.0508	0.9990
3	2	0.0061	0.0261	0.9990
0	3	0.6960	0.0039	0.4830
1	3	0.0008	0.0033	1.0000
2	3	0.0016	0.0057	0.9995
3	3	0.0000	0.0000	1.0000

3.3 Question 3

- *Question_1* Brief interpretation of the effect of changing the depth and width on the quality of classification in the above experiment:

Findings:

- For neural networks of the same depth (except for no hidden layers), increasing the width of the network lead to lower mean test loss and possibly higher mean test accuracy in our experiment.
- For neural networks of the same width (except for no hidden layers and $width = 1$), increasing depth lead to a lower mean test loss.

Conclusion:

Increasing depth and width individually does not lead to lower test loss and higher test accuracy. We assume that increasing the depth of a neural network only improves the network's ability to capture complicated patterns, likely because increasing the width only represents the feature size (features of the input) of the hidden layer, that is, how complex the pattern is. Conversely, increasing solely the width leads to an increase in pattern complexity. If the depth of a neural network is low, the network has limited ability to solve complicated patterns.

As a result, increasing both depth and width at the same time improves the fitting ability of the network, which potentially leads to higher quality of classification.

- *Question_2* Calculate how many parameters each of the networks tested above:

The number of parameters for each network are shown on the Table 3.

Explanation on how we get the parameter numbers of each network:

- We use the following function to extract the number of parameters for each neural network:

```
sum(p.numel() for p in model.parameters() if p.requires_grad)
```

- We use the `summary` function from the `torchsummary` package to cross check the parameter number for each network.

```
torchsummary.summary(model, input_size=(2,))
```

- *Question_3 Identify the training parameters used, and describe how changing them, may influence your conclusions on the quality of the classification:*

- What is training parameters?

We use the model from question 1 as baseline model and alter its training parameters individually to see how things change accordingly. The baseline model for the training parameters is configured as follows:

- * **Learning Rate:** 0.01
- * **Optimizer:** Adam
- * **Batch Size:** 10
- * **Epochs:** 20

- How does changing the parameters affect the quality of the classification?

- * **Optimizer:** change from "Adam" to "SGD":
It does not affect the classification.

- * **Learning rate:** change from "0.01" to "0.1", "0.5", "0.8" and "0.001":
Test accuracy still around 100% when learning rate = 0.1 and 0.5. When increasing the learning rate to 0.8, the average test loss for each batch in the test set increases, and test accuracy reaches 75%. Meanwhile, for a learning rate of 0.001, the average test loss for each batch in test set increases, and test accuracy also reaches 75%. So increasing learning rate and decreasing learning rate to a certain level would cause decrease the fitting quality of the neural network:

- With a small learning rate, the network needs more steps to converge. If there are not enough epoch, the test loss is high, and test accuracy is low.
- With a high learning rate, the network misses the local minimum during the parameters update step. Consequently the test loss is high, and test accuracy low.

- * **Epoch** - changing the number of epochs from "20" to "1000", and "3":
If we set a higher epoch number (like 1000), the test accuracy is still 100% as the original model with 20 epochs. However, the validation loss begins to increase from 0 to 0.03 at the 250th epoch which is a sign of over-fitting. For only 3 epochs, the average test loss is around 0.59 and the test accuracy is 82%.

- * batch size - changing batch size from "10" to "100" and "1"

When increasing the batch size from 10 to 100, the test accuracy drops from 100% to 98%. We believe that may be because bigger batch size means a lower number of batches to iterate at each epoch. So the parameter update become faster which leads to sharper minima in the loss landscape. It may, however, not generalize well on the test dataset because the network only learns the "average of each batch" but not the uniqueness of the data.

Small batch sizes - for example only 1 batch - showed the lowest test loss and 100% of test accuracy. However, validation loss drops faster than train loss because small batch sizes introduce more noise to the model. The model is actually fitting both the data and their noise which we see with the training loss and validation loss hitting around 0 after several epochs.

Table 3: Summary of Mean Test Metrics (Sorted by Parameters)

Depth	Width	Parameters	Mean Test Loss	Std Dev	Mean Test Accuracy
0	1	3	0.6970	0.0042	0.4620
0	2	3	0.6994	0.0054	0.4695
0	3	3	0.6960	0.0039	0.4830
1	1	5	0.4716	0.0419	0.7580
2	1	7	0.4826	0.0252	0.7435
1	2	9	0.3547	0.0229	0.7355
3	1	9	0.4836	0.0372	0.7555
1	3	13	0.0008	0.0033	1.0000
2	2	15	0.0126	0.0508	0.9990
3	2	21	0.0061	0.0261	0.9990
2	3	25	0.0016	0.0057	0.9995
3	3	37	0.0000	0.0000	1.0000

4 MLOps

All the code in feedforwardAssignment.ipynb

4.1 Question 1

- Promising network configuration:
One hidden layer with width = 3. This is mainly because this structure is the simplest structure that achieves 100% accuracy on the test set.
- Resulting plots can be found in Figure 6 on the next page.
- Modification
Parameter concerning definition:

```
run = wandb.init(project='Other')
config = wandb.config
config.learning_rate = 0.01
config.epochs = 40
optimizer = torch.optim.Adam(model.parameters(), lr=config.
    learning_rate)
```

Parameter concerning training:

```
for epoch in range(config.epochs)
```

Parameter concerning printing and tracking:

```
wandb.log({
    'epoch': epoch + 1,
    'train_loss': train_output[0],
    'train_accuracy': train_output[1],
    'val_loss': val_output[0],
    'val_accuracy': val_output[1]
})
wandb.log({
    'epoch': epoch + 1,
    'batch_loss': running_loss / 2000
})
wandb.log({'epoch': epoch + 1, 'learning_rate': current_lr})
```

4.2 Question 2 and Question 3

For these tasks, we increase the noise s of our generated data from 0.1 to 0.5. And we want to use a small experiment to explore which basic setting should be optimal to get a higher test accuracy and still avoid over-fitting and under-fitting. Also, we don't want to change the network structure to explore whether changing only training parameters can affect learning performance of neural network.



Figure 6: Log plots from Weight and bias

Firstly, we think the important parameters are as follows:

- batch size
- number of epochs
- learning rate
- activation functions used in hidden layers: `relu`, `tanh`
- optimizer function: `Adam`, `sgd`

Experimental setup: Initially, we run a baseline model using the below setting for 20 times.

```
import math
sweep_config = {
    'method': 'random',
    'metric': {
        'name': 'test_accuracy',
        'goal': 'maximize',
    },
    'parameters': {
        'noise': {'value': 0.5},
        'input_dim': {'value': 2},
        'output_dim': {'value': 1},
        'hidden_dim': {'value': 3},
        'n_hidden_layers': {'value': 1},
        'activation': {'value': "tanh"},
        'optimizer': {'value': "adam"},
        'learning_rate': {'value': 0.01},
        'batch_size': {'value': 10},
        'epochs': {'value': 20},
    },
}
```

```
}
}
```

Visualization of the results are show in figure 7.

We found that the test accuracy ranges from 60% to 81%. We also cross-checked the loss and accuracy from the validation set and the training set in figure 8 to ensure the model does not over-fit or under-fit (the gradient of the validation loss and training loss becomes saturated and approaches zero at a certain step/epoch ($step : epoch = 1 : 1$)). On the basis of this, we conclude that our baseline model has a mean test accuracy between 70% and 80%.

Now we want to find the optimal setting for our network to increase the accuracy on the test set. Normally, we would only have to change several parameters of the model and test for several times to get the best average model, then perform this repeatedly until we find the best model. Due to limited computing resources, however, we could only perform tests with several instances of certain parameters. The setting of sweep configure show below:

```
import math
sweep_config = {
    'method': 'random',
    'metric': {
        'name': 'test_accuracy',
        'goal': 'maximize',
    },
    'parameters': {
        'noise': {'value': 0.5},
        'input_dim': {'value': 2},
        'output_dim': {'value': 1},
        'hidden_dim': {'value': 3},
        'n_hidden_layers': {'value': 1},
        'activation': {'values': ["tanh", "relu"]},
        'optimizer': {'values': ["adam", "sgd"]},
        'learning_rate': {
            'distribution': 'log_uniform',
            'min': math.log(0.0001),
            'max': math.log(0.1)
        },
        'batch_size': {
            'values': [8, 16, 32, 64]
        },
        'epochs': {
            'values': [50, 100, 150]
        }
    }
}
```

We randomly test these settings 100 times. The preliminary results are show in figure 9. Firstly, we avoid over-fitting and under-fitting by looking into their training loss and validation loss line chart. Then we list their setting in the below table 4:

At this point, we run the model with the best test accuracy 10 times. We find that we could not improve on the results, which suggests that the model we generated in the previous experiment was probably by chance.

We then tried a more extreme model with `batch_size = 1`, `learning_rate = 0.001`, `epoch = 1000` 10 times. This configuration did not produce better test accuracy, either. Also, during training, the model's training loss plateaued at a certain point and did not decrease again.

Table 4: Model Parameters and Test Accuracy

Batch Size	Epochs	Learning Rate	Activation Function	Optimizer	Test Accuracy
32	100	0.0016	tanh	Adam	0.84
64	150	0.05	relu	sgd	0.82
32	150	0.007	tanh	Adam	0.82
8	50	0.002	tanh	Adam	0.80

On the basis of these findings, our preliminary conclusion is that adding noise to the data increases the complicity of the data (increases the number of features). Changing the training parameters like learning rate, epoch number and batch size might not necessarily increase the model's performance on the test set. Another reason is that the data points on x_1 and x_2 mix with each other when adding too much noise, and the data loses its general pattern. That means if we set $s = 1$, the data points might be randomly distributed in 2D space restricted by

$$x_1 \in [-1, 2]$$

and

$$x_2 \in [-1, 2]$$

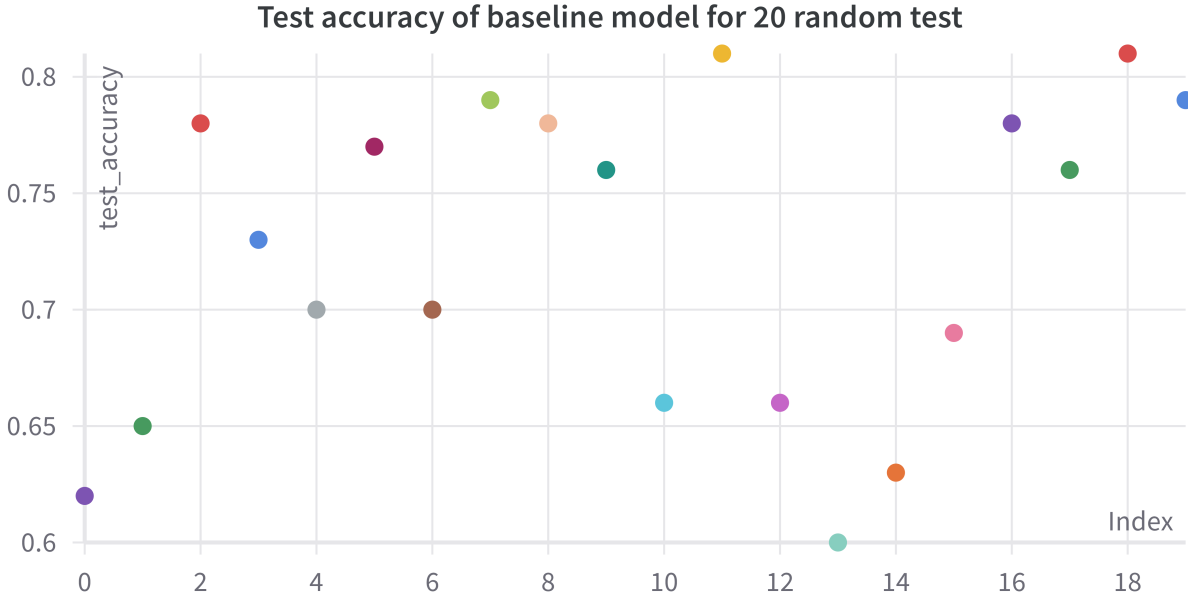


Figure 7: Plots of test accuracy for baseline models in 20 independent tests

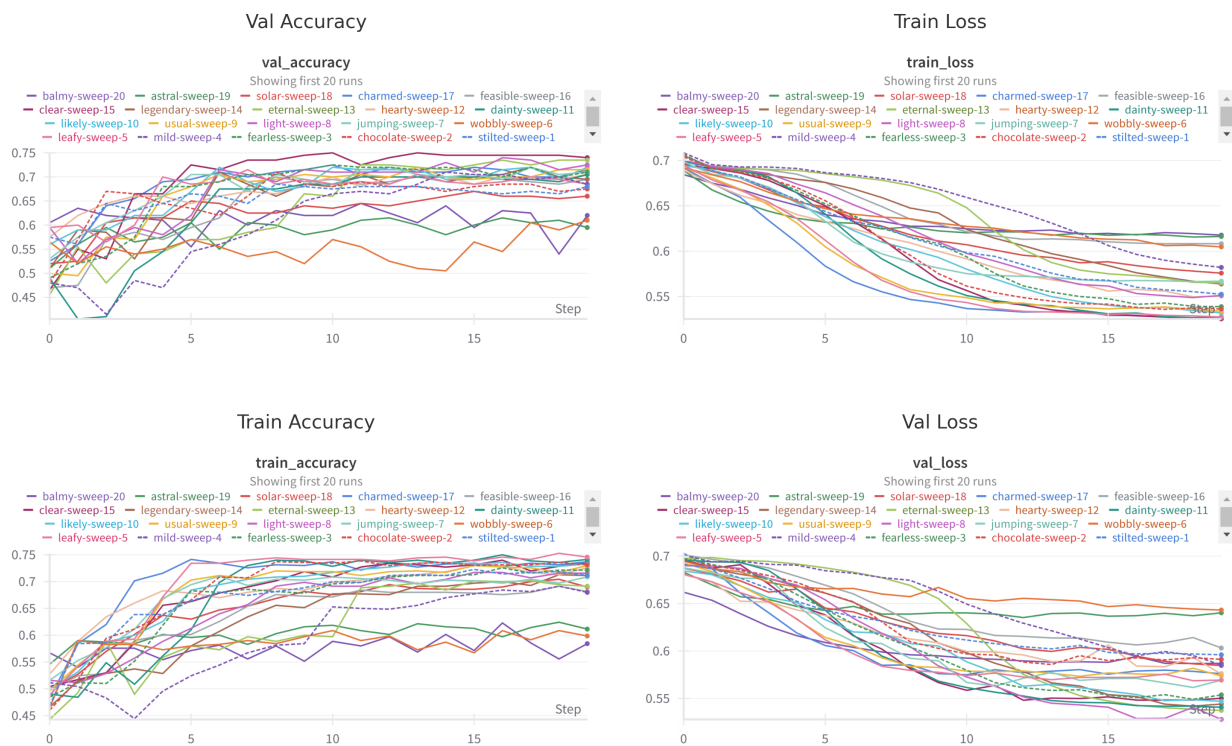


Figure 8: Plots of loss and accuracy from training set and validation set for baseline models in 20 independent tests

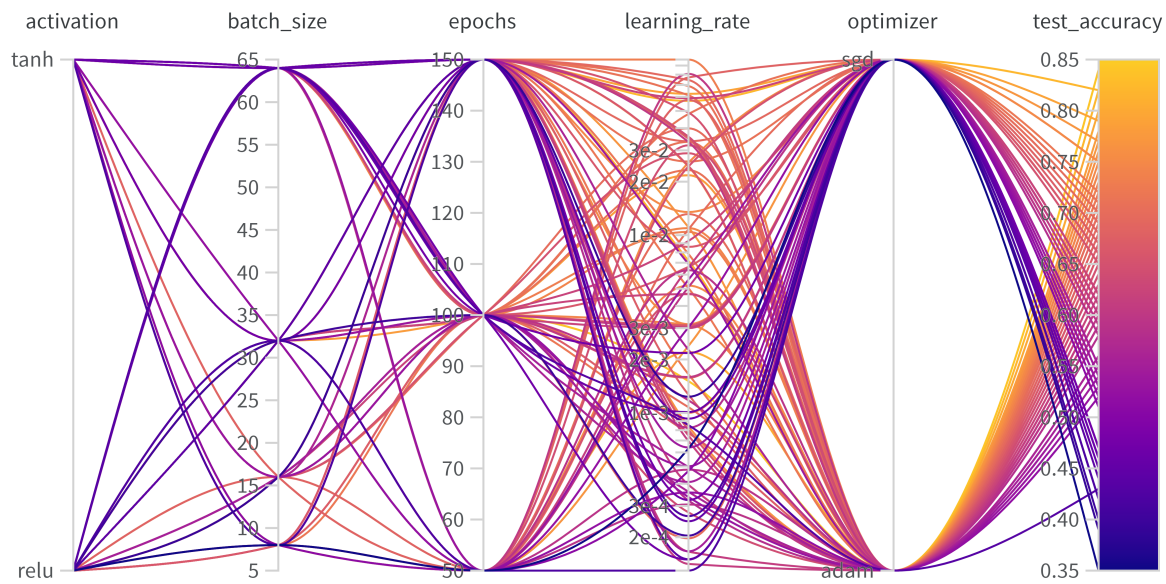


Figure 9: parallel coordinate plot for random setting